TEK-UP
Ecole Supérieure Privée
Technologie & Ingénierie

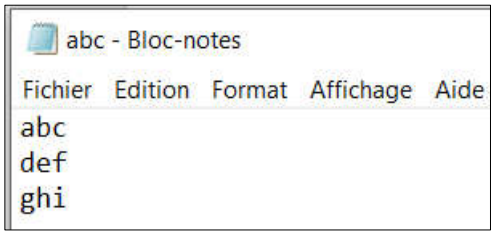
Ecole Supérieure Privée Technologies & Ingénierie			
Type d'épreuve	: ☐ Devoir	■ Examen SESSION PRINCIPALE	
Matière	: Programmation Python		
Année Universitaire	: 2024-2025	**Semestre**	: 2
Classes	: ING-3-J		
Documents	: ☐ Autorisés	■ Non autorisés	
Date	: 29/05/2025	**Durée :**	1h30mn
Nombre de pages	: 4 pages		
Barème	: 5 pts + 5 pts +10 pts		

Exercice 1 (5 pts) : pour chaque fragment de code, donnez la sortie affichée par le programme une fois exécuté.

1. Code 1 : **(1 pt)**

```
file = open('abc.txt')
a = [i for i in file]
print(a)
```

Le fichier **abc.txt** contient les 3 lignes de texte suivantes :



2. Code 2 : **(1 pt)**

```
a = "ab cd"
print('*'.join(a.split()))
```

3. Code 3 : **(0.5 pt)**

```
a = "abcdef"
a[1] = 'x'
print(a)
```

4. Code 4 : **(0.5 pt)**

```
mois=['Mai', 'Juin', 'Juillet']
print(mois[-2][-2])
```

5. Code 5 : **(1 pt)**

```
a = [1,2,3,4,5,6,7,8,9]
b = [i if i%3==0 else 0 for i in a]
print(b)
```

6. Code 6 : **(1 pt)**

```
print((lambda x: x[::-1].upper())("bonjour"))
```

Exercice 2 : (5 pts)

Nous nous intéressons à la gestion d'une base de données regroupant des salariés d'une entreprise. Le programme doit permettre d'ajouter des salariés, de rechercher leurs informations, et de réaliser des calculs simples. Chaque salarié est représenté par un dictionnaire contenant les informations suivantes :

- nom : (str)
- email : (str)
- projets : (liste de tuples de taille 3) – chaque tuple contient :
    - nom_projet (str)
    - heures_travaillées (float)
    - taux_horaire (float)

Les informations des salariés sont stockées dans une liste **BD_salaries.**

1. Définissez une fonction **ajouter_salarie()** qui saisit les informations d'un salarié, en effectuant les contrôles de saisie nécessaires, et l'ajoute à la liste **BD_salaries. (1.5 pt)**
2. Définissez une fonction **salaire_total()** qui, pour un salarié donné par son email, calcule et retourne le total de son salaire, calculé en multipliant les heures travaillées par le taux horaire sur tous ses projets. **(1.5 pt)**
3. Définissez une fonction **classement_salaries()** qui trie, par ordre décroissant, les salariés selon les heures travaillées sur **un projet donné (par son nom)**. **(2 pt)**

Exercice 3 : (10 pts)

Dans le cadre du développement d'un système de gestion pour un **parking automobile**, on souhaite modéliser les entités nécessaires à la gestion des véhicules et des places disponibles. Vous êtes invités à concevoir un ensemble de classes permettant de représenter cette situation de manière orientée objet.

1. **Modélisation de la classe Voiture :** Chaque voiture est caractérisée par les attributs suivants :
    - immatriculation (str) : le numéro d'immatriculation de la voiture,
    - est_garee (bool) : un indicateur précisant si la voiture est actuellement garée dans le parking.

Par défaut, une voiture nouvellement créée est considérée **non garée**.

a. Définissez la classe Voiture en prévoyant un **constructeur** qui demande à l'utilisateur de saisir l'immatriculation du véhicule lors de la création d'un objet. (**0.75 pt**)

b. Définissez une méthode **afficher()** qui affiche les informations de la voiture selon le format suivant : (**0.75 pt**)

Immatriculation : 123-TN-456

État : Non garée

2. **Modélisation de la classe Parking :** Nous souhaitons modéliser un parking capable d'accueillir un nombre limité de voitures. Pour cela, nous définissons une classe Parking caractérisée par les attributs suivants :
   - nom (str) : le nom du parking,
   - capacite (int) : la capacité maximale du parking,
   - voitures (list) : une **liste privée**, **initialement vide**, contenant les objets Voiture actuellement garés dans le parking.

a. Définissez la classe Parking en prévoyant un **constructeur paramétré**. (**1 pt**)

b. Implémentez la méthode **afficher_parking()** qui affiche le nom et la capacité du parking. (**1 pt**)

c. Implémentez la méthode **afficher_voitures()** qui affiche les informations de toutes les voitures actuellement garées dans le parking. (**1 pt**)

d. Implémentez la méthode **ajouter_voiture(),** qui étant donné le numéro d'immatriculation, crée un objet Voiture puis l'ajoute au parking **uniquement si une place est disponible**. Sinon, afficher le message : « Parking Plein » (**1.25 pt**)

e. Implémentez la méthode **sortir_voiture**() qui modifie l'état de la voiture correspondant à l'immatriculation donnée afin d'indiquer qu'elle a quitté le parking. Si aucune voiture portant cette immatriculation n'est trouvée, la méthode doit afficher : « Voiture introuvable ». (**1.25 pt**)

3. **Implémentation du programme principal :** Développez un script permettant de :
   a. Demander à l'utilisateur combien de parkings il souhaite gérer. (**0.25 pt**)
   b. Instancier ces objets Parking et les stocker dans un dictionnaire **dict[str, Parking]**, où : les **clés** sont les noms des parkings, les **valeurs** sont les objets Parking correspondants. (**1.5 pt**)
   c. Demander à l'utilisateur de saisir un nom de parking et une immatriculation de voiture. Si le parking existe, ajouter la voiture correspondante et afficher les véhicules qui y sont garés ; sinon, afficher : « Ce parking n'existe pas. » (**1.25 pt**)